

Semantic Search

Project Status Report | Last updated: July 9, 2026

The Big Picture

All the code for smarter, meaning-based search (on top of the existing keyword search) is written, tested, and confirmed to work with your real Voyage AI account and real database. **Nothing is live for customers yet.** There is exactly **one thing** stopping it from going live: a database permission. The current database login can only *read* data, not *save* new data — and this feature needs to save a small amount of new data onto each product. Once that permission is switched on, the rest of the work is a short, already-planned checklist — not new design work.

THE ONE BLOCKER

The database login (`rutujazatch_db_user`) is read-only. This feature needs write access so it can save a small "embedding" (a numeric fingerprint of the product's meaning) onto each product and seller.

Fix: Rakshit (or whoever owns the Atlas database) needs to change that login's permission from read-only to read-and-write. This takes under a minute and needs no restart.

Status at a Glance

Here is every step in the project, in order, and where each one currently stands.

#	Step	Status	Notes
1	Write the core code (search logic, meaning-matching, combining results, safe fallbacks)	■ Done	—
2	Write the supporting scripts (one-time setup, testing tools)	■ Done	—
3	Update project configuration files	■ Done	—
4	Test everything on a fake/local copy of the data	■ Done	—
5	Confirm it connects to your real database (read 141 products, 246 users)	■ Done	—

#	Step	Status	Notes
6	Confirm it connects to your real AI provider (Voyage AI)	■ Done	—
7	Save the AI "fingerprints" onto real products	■ Blocked	Needs write access (see above)
8	Create 2 search-speed-up indexes in the database	■ Optional	Optional — has a working backup plan
9	Test the whole thing live, end to end	■ Waiting	Waiting on step 7
10	Fine-tune accuracy using real test questions	■ Waiting	Waiting on step 9
11	Launch it for real (deploy)	■ Waiting	Waiting on step 7

■ Done = finished and verified
 ■ Blocked = stuck until someone acts
 ■ Optional = nice to have, not required to launch
 ■ Waiting = next in line

What's Already Done

The Code

Everything needed to make search understand *meaning*, not just exact keywords, has been written and tested:

- A tool that turns product/seller text into an AI "fingerprint" (a list of 1,024 numbers that represents its meaning), using Voyage AI.
- A search tool that finds products with a similar meaning to what the customer typed — and automatically falls back to a slower-but-safe method if the fast database index isn't ready yet.
- A merging tool that blends classic keyword search with the new meaning-based search into one ranked list.
- The main search function now runs both methods together, with a strict 2-second time limit on the AI call — so search can never freeze or crash, even if the AI service is slow.
- A one-time script to fill in the AI fingerprints for all existing products and sellers.
- A script to set up the two database search indexes.
- Automated tests (all passing) and a scoring tool to measure search accuracy later.

Configuration & Setup

- Project settings files were updated with the new AI service key and a new background job that keeps fingerprints fresh every 6 hours.
- Package versions were adjusted where the originally planned versions didn't actually install.
- Old, no-longer-needed code was cleaned up.

How It Was Tested

This wasn't just tested in theory — it was tested against your real systems:

- **Code correctness:** every file compiles with no errors; all automated tests pass.
- **Simulated tests:** tried three scenarios on a practice database — no AI key configured, the AI service failing mid-request, and the AI service working normally. In every case, search kept working and never crashed.
- **Real database:** successfully connected and read 141 real products and 246 real users.
- **Real AI service:** successfully asked Voyage AI to "fingerprint" the phrase "wedding outfit" and got back a real result.

In short: this has all been proven against your actual data and actual AI account — it just hasn't been given permission to save anything yet.

What's Left, and How to Finish It

1. Give the database login write access

MUST DO — BLOCKS
EVERYTHING

The problem: the one-time script that saves AI fingerprints tried to run, and the database rejected it because the login can't make changes — only read.

The fix: someone with admin rights on the database (Rakshit) needs to change the login's permission level from read-only to read-and-write. This is a settings change in the database's website — no code involved.

Steps:

- 1 Log into Atlas (the database's website) with an account that has admin rights.
- 2 Go to Database Access → find the login named **rutujazatch_db_user** → click Edit.
- 3 Change its permission to "Read and write to any database" (or just to the **zatch** database).
- 4 Save. It takes effect in under a minute — no restart needed.
- 5 Let me know it's done, and I'll immediately run the save-fingerprints step — for the current amount of data (141 products + a couple hundred sellers), that takes well under a minute and costs effectively nothing.

Faster alternative: if that's slow to arrange, sharing the login details for the existing **zatch-app** user (which already has full write access) would work instantly, with no database settings to change at all.

2. Create the two speed-up search indexes

OPTIONAL — NOT
BLOCKING LAUNCH

What this is: a database index is like a table of contents — it makes searches faster. Creating one needs a different, higher level of admin permission than saving data does.

Why it's okay to skip for now: the search code already has a built-in backup plan — if the fast index isn't there yet, it calculates the same results a slightly slower way in the background. Customers get identical, correct results either way. With only 141 products, the speed difference is not noticeable.

When convenient, either give the Atlas login "Project Owner" rights, or have Rakshit create the two indexes directly using the settings provided in the technical appendix at the end of this document. Once created, the system starts using them automatically — no code changes needed.

3. Run the save-fingerprints script

WAITING ON STEP 1

Once write access is granted, this one command computes and saves an AI fingerprint for every product and seller. Expected time: a few seconds, given the current catalog size.

```
python scripts/backfill_embeddings.py
```

4. Test it live against your real catalog

WAITING ON STEP 3

Try a handful of searches that keyword-only search struggles with today — things like "wedding outfit", "gift for wife", or "shop that sells sarees" — and check whether the results make sense for your actual products. This is the first real look at how well the AI performs on your specific catalog; some fine-tuning afterward is normal.

5. Fine-tune accuracy

WAITING ON STEP 4

There's a scoring tool with a list of test questions, currently empty. Once step 4 shows which products *should* match each test question, that list gets filled in and the scoring tool is run to measure accuracy — then small settings get adjusted to improve it (how strict the AI matching is, how results are ranked, etc.).

```
python tests/run_eval.py
```

6. Launch it for real (deploy)

WAITING ON STEP 1

The hosting configuration (Render) is already updated and ready. To actually go live:

- 1 In the Render dashboard, set the database link and the Voyage AI key on both the main web service and the new background "keep fingerprints fresh" job.
- 2 Push the update — Render installs everything and starts both services automatically.
- 3 Check Render's logs to confirm the background job's first run succeeds.

7. Optional cleanup, later

LOW PRIORITY

Once semantic search is live and proven, a large list of manual typo-corrections in the code can be shrunk — most of it becomes unnecessary once the AI understands similar words on its own.

Who Needs to Do What

A quick summary of every outstanding request:

What's needed	From whom	Why
Change the database login's permission to read-and-write (or share working write-access credentials)	Rakshit / database admin	Unblocks everything — step 1
Either give write-access to create search indexes, or create the 2 indexes directly (details in appendix)	Rakshit / database admin	Optional — improves speed at larger scale
A few real product/seller names that should match specific test searches	You, after reviewing step 4's results	Needed to fine-tune accuracy
Database link + AI key entered into the Render hosting dashboard	You	Needed to launch

NEXT ACTION

The single next action, for whoever has admin rights on the database, is: **change rutujazatch_db_user's role to "read and write" on the zatch database** (Atlas → Database Access → Edit). Everything else in this report is either already done, or waiting directly on that one change.

Appendix: Technical Reference

A. Files Involved

For reference, this is the full list of files that were added or changed:

File	Purpose
<code>render.yaml</code>	Hosting config — added AI key + background job
<code>requirements.txt</code>	Package list — fixed encoding, added AI + math packages
<code>.env.example</code> / <code>.env</code>	New — stores credentials (kept private)
<code>app/search.py</code>	Main search logic — now combines keyword + meaning-based search
<code>app/embeddings.py</code>	New — talks to Voyage AI, builds text for each product/seller
<code>app/vector_search.py</code>	New — meaning-based search, with automatic backup method
<code>app/hybrid.py</code>	New — blends the two search methods into one ranked list
<code>scripts/backfill_embeddings.py</code>	New — one-time job to save AI fingerprints
<code>scripts/create_vector_indexes.py</code>	New — one-time job to create the 2 speed indexes
<code>tests/test_embeddings.py</code>	New — automated tests (all passing)
<code>tests/golden_queries.json</code> + <code>run_eval.py</code>	New — accuracy scoring tool

B. Search Index Settings (for step 2 above)

For whoever ends up creating the two optional search indexes in Atlas (cluster → Atlas Search tab → Create Search Index → JSON Editor):

Index 1 — database `zatch`, collection `products`, name `product_vector_index`:

```
{
  "fields": [
    { "type": "vector", "path": "embedding", "numDimensions": 1024, "similarity": "cosine" },
    { "type": "filter", "path": "status" },
    { "type": "filter", "path": "discountedPrice" }
  ]
}
```

Index 2 — database zatch, collection users, name seller_vector_index:

```
{
  "fields": [
    { "type": "vector", "path": "embedding", "numDimensions": 1024, "similarity": "cosine"
  }
  ]
}
```

Both take about a minute to finish building. The database plan in use allows up to 3 indexes and is currently using 1, so there's room for both.